Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Chapter 5
Orthogonal methods

The most reliable methods for solving least squares problems use orthogonal transformations. This chapter considers QR factorization based on Householder reflections and Givens rotations.

5.1 Householder reflections

A *Householder reflection* is an orthogonal matrix of the form $H = I - \beta vv^T$, where β is a scalar and v is a column vector. The vector v and scalar β can be chosen based on a vector x so that Hx is zero except for the first entry $(Hx)_1 = \pm\|x\|_2$. The MATLAB function `[v,beta,s]=gallery('house',x,2)` computes `v`, `beta`, and `s` $= (Hx)_1$. Applying H to a vector x of length n takes only about $4n$ floating-point operations ($Hx = x - v(\beta(v^T x))$). The matrix H is symmetric and orthogonal ($HH^T = H^T H = I$), as a consequence of how β and v are chosen. The `cs_house` function ensures `s` $= \|x\|_2$. It computes β and `s` and overwrites `x` with `v`.

```
double cs_house (double *x, double *beta, int n)
{
    double s, sigma = 0 ;
    int i ;
    if (!x || !beta) return (-1) ;          /* check inputs */
    for (i = 1 ; i < n ; i++) sigma += x [i] * x [i] ;
    if (sigma == 0)
    {
        s = fabs (x [0]) ;                  /* s = |x(0)| */
        (*beta) = (x [0] <= 0) ? 2 : 0 ;
        x [0] = 1 ;
    }
    else
    {
        s = sqrt (x [0] * x [0] + sigma) ;  /* s = norm (x) */
        x [0] = (x [0] <= 0) ? (x [0] - s) : (-sigma / (x [0] + s)) ;
        (*beta) = -1. / (s * x [0]) ;
    }
    return (s) ;
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Let $\mathcal{V}$ and $\mathcal{X}$ be the nonzero pattern of v and x, respectively. If $x_1 \neq 0$, then $\mathcal{V} = \mathcal{X}$. The theorems and algorithms for the sparse QR factorization are simplified if this condition always holds, and thus this discussion assumes that x_1 is a structural entry in x. That is, x_1 will always be an entry in the sparse vector `x`, but it might be numerically zero. Then, ignoring numerical cancellation, $\mathcal{V} = \mathcal{X}$ is always true. Normally, a Householder reflection H is applied not only to the vector x it was created from but also to other vectors or matrices as well. If applied to a vector y ($\overline{y} = Hy$), and assuming $x_1 \neq 0$, the nonzero pattern of $\overline{y}$ becomes $\overline{\mathcal{Y}} = \mathcal{Y} \cup \mathcal{V} = \mathcal{Y} \cup \mathcal{X}$ if $\mathcal{Y} \cap \mathcal{X}$ is not empty, and $\overline{\mathcal{Y}} = \mathcal{Y}$ otherwise.

The function `cs_happly` applies a Householder reflection to a dense vector `x`, where `v` is sparse. It overwrites `x` with `x-v*(beta*(v'*x))`. The vector `v` is obtained from the `i`th column of the matrix `V`.

```
int cs_happly (const cs *V, int i, double beta, double *x)
{
    int p, *Vp, *Vi ;
    double *Vx, tau = 0 ;
    if (!CS_CSC (V) || !x) return (0) ;     /* check inputs */
    Vp = V->p ; Vi = V->i ; Vx = V->x ;
    for (p = Vp [i] ; p < Vp [i+1] ; p++)   /* tau = v'*x */
    {
        tau += Vx [p] * x [Vi [p]] ;
    }
    tau *= beta ;                           /* tau = beta*(v'*x) */
    for (p = Vp [i] ; p < Vp [i+1] ; p++)   /* x = x - v*tau */
    {
        x [Vi [p]] -= Vx [p] * tau ;
    }
    return (1) ;
}
```

5.2 Left- and right-looking QR factorization

In this section, two Householder-based QR factorization algorithms are described that factorize the m-by-n matrix A into the product QR, where Q is orthogonal and R is upper triangular (or upper trapezoidal if $m < n$): a *left-looking* algorithm and a *right-looking* algorithm.

Suppose A is m-by-n with $m \geq n$. A sequence of n Householder reflections $H_1, \ldots, H_n$ is chosen to reduce A to upper triangular form. Let $A^{[k]}$ denote the product $H_k H_{k-1} \ldots H_1 A$. The first Householder reflection, H_1, is constructed from the first column of A; thus, the first column of $H_1 A$ is all zero except for the diagonal entry, $a_{11} = \|A_{*1}\|_2$. The kth Householder reflection is chosen based on $x = A^{[k-1]}_{k \ldots m,k}$, a vector of length $m-k+1$, resulting in β_k (a scalar) and $\overline{v}_k$ (a vector of length $m-k+1$). Let v_k be a column vector of length m that is zero in rows 1 to $k-1$ and $(v_k)_{k \ldots m} = \overline{v}_k$. Then $H_k = I - \beta_k v_k v_k^T$.

Theorem 5.1 (Golub [113]). *The QR factorization $QR = A$, where $A \in \mathbb{R}^{m \times n}$ and $m \geq n$, is $Q = H_1 H_2 \cdots H_n = \prod_{k=1}^{n} H_k$ and $R = Q^T A = H_n \cdots H_2 H_1 A = (\prod_{k=n}^{1} H_k) A = A^{[n]}$.*

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

The sequence of Householder reflections can be applied in a left-looking or right-looking manner. The right-looking algorithm `qr_right` simply applies each Householder reflection to all of `A` as soon as it is constructed. The `qr_right` function is difficult to implement in a concise sparse matrix algorithm, although it forms the basis of the multifrontal sparse QR method.

```
function [V,Beta,R] = qr_right (A)
[m n] = size (A) ;
V = zeros (m,n) ;
Beta = zeros (1,n) ;
for k = 1:n
    [v,beta,s] = gallery ('house', A (k:m,k), 2) ;
    V (k:m,k) = v ;
    Beta (k) = beta ;
    A (k:m,k:n) = A (k:m,k:n) - v * (beta * (v' * A (k:m,k:n))) ;
end
R = A ;
```

The left-looking algorithm `qr_left` applies the Householder reflections only to the current column `k`, one column at a time, and is simpler to implement for the sparse case.

```
function [V,Beta,R] = qr_left (A)
[m n] = size (A) ;
V = zeros (m,n) ;
Beta = zeros (1,n) ;
R = zeros (m,n) ;
for k = 1:n
    x = A (:,k) ;
    for i = 1:k-1
        v = V (i:m,i) ;
        beta = Beta (i) ;
        x (i:m) = x (i:m) - v * (beta * (v' * x (i:m))) ;
    end
    [v,beta,s] = gallery ('house', x (k:m), 2) ;
    V (k:m,k) = v ;
    Beta (k) = beta ;
    R (1:(k-1),k) = x (1:(k-1)) ;
    R (k,k) = s ;
end
```

Note that `qr_right` and `qr_left` do not compute an explicit representation of Q; this is also true of the sparse left-looking QR factorization described next.

5.3 Householder-based sparse QR factorization

The left-looking QR factorization algorithm (`qr_left`) forms the basis of the sparse QR factorization algorithm presented below.

Let $\mathcal{R}_{i*}$ and $\mathcal{R}_{*j}$ be the nonzero pattern of row i and column j of R, respectively. Let $\mathcal{A}_{i*}^{[k]}$ and $\mathcal{A}_{*j}^{[k]}$ denote the nonzero pattern of row i and column j of $A^{[k]}$, respectively. Let $\mathcal{V}_k$ be the nonzero pattern of v_k. Let $\mathcal{T}$ be the column elimination tree of A (the elimination tree of $A^T A$).

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Some of the theorems stated here require $A_{kk}^{[k-1]}$ to be structurally nonzero (that is, it is an entry in the data structure even if numerically zero). If this is not the case, then the rows of A can be permuted, or the sparse matrix `A` can be modified by adding explicit zero entries, to ensure that this condition holds. All of the theorems ignore numerical cancellation. Some theorems require the matrix to have the *strong Hall property*; they provide loose upper bounds on the nonzero pattern otherwise. Because of space constraints, the proofs are brief or omitted. The definition of *strong Hall* is given in Section 7.3.

Theorem 5.2 (George, Liu, and Ng [95]**).** *Consider $HA = A - v(\beta(v^T A))$. Then $(HA)_{i*}$, where $i \notin \mathcal{V}$ is equal to row i of A. For any row $i \in \mathcal{V}$, the nonzero pattern of $(HA)_{i*}$ is*

$$\bigcup_{i \in \mathcal{V}} \mathcal{A}_{i*}. \tag{5.1}$$

That is, in HA, the nonzero pattern of any modified row $i \in \mathcal{V}$ is replaced with the set union of all rows that are modified by the Householder reflection H.

Proof. From Theorem 2.1, the nonzero pattern of $v^T A = (A^T v)^T$ is given by (5.1). The entry $(v(v^T A))_{ij}$ (β can be ignored) is nonzero if and only if $i \in \mathcal{V}$ and $j \in \cup_{i \in \mathcal{V}} \mathcal{A}_{i*}$. The matrix $v(v^T A)$ is then subtracted from A to obtain HA, modifying all rows $i \in \mathcal{V}$. Each of the corresponding rows of A was used to construct the set union (5.1), so all of them now have the same nonzero pattern, given by (5.1). ∎

Theorem 5.3 (Golub and Van Loan [114]**).** *If $A^T A$ is positive definite, and its Cholesky factorization is $LL^T = A^T A$, then $L = R^T_{1\ldots n, 1\ldots n}$.*

Proof. $A^T A = (QR)^T QR = R^T Q^T QR = R^T R.$ ∎

Theorem 5.4 (George and Ng [97]**).** *If $A_{kk}^{[k-1]}$ is structurally nonzero for all $1 \le k \le n$, then $\mathcal{R}_{i*} = \mathcal{A}_{i*}^{[k]}$.*

Theorem 5.5 (Coleman, Edenbrandt, and Gilbert [22], George and Heath [83]**).** *Assuming the matrix A has the strong Hall property, $\mathcal{R}_{*k} = \mathcal{L}_{k*}$, where $\mathcal{L}_{k*}$ denotes the nonzero pattern of the kth row of the symbolic Cholesky factor of $A^T A$. If A does not have the strong Hall property, $\mathcal{R}_{*k} \subseteq \mathcal{L}_{k*}$.*

Corollary 5.6. *$\mathcal{R}_{*k} = Reach_{\mathcal{T}_k}(\mathcal{C}_k)$, where $\mathcal{C}_k$ is the nonzero pattern of the upper triangular part of column k of $A^T A$ (assuming A has the strong Hall property).*

A more concise method for computing $\mathcal{R}_{*k}$ is based on the following theorem.

Theorem 5.7. *$\mathcal{R}_{*k} = Reach_{\mathcal{T}_k}(\{\min \mathcal{A}_{i*} \mid i \in \mathcal{A}_{*k}\})$ (assuming A has the strong*

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Hall property).

Proof. Consider column k of A and the nonzero pattern of $C = A^T A$. The matrix C can be written as a summation of outer products, one for each row, $C = \sum_{i=1}^{m} A_{i*}^T A_{i*}$. Consider the entry $a_{ik} \neq 0$ in column k. The nonzero pattern of $A_{i*}^T A_{i*}$ is a dense submatrix of C; every c_{k_1,k_2} is nonzero for any pair of column indices k_1 and k_2 in $\mathcal{A}_{i*}$. From Theorem 4.13, $k_1 < k_2$ implies $k_1 \rightsquigarrow k_2$ in $\mathcal{T}_k$. Thus, k_2 is redundant for computing $\mathrm{Reach}_{\mathcal{T}_k}(\mathcal{C}_k)$. Only $j = \min \mathcal{A}_{i*}$, the smallest column index of nonzero entries in row i, needs to be considered for any entry a_{ik} in column k of A. This j corresponds to the leftmost entry a_{ij} in row i of A. Thus $\mathcal{R}_{*k} = \mathrm{Reach}_{\mathcal{T}_k}(\mathcal{C}_k) = \mathrm{Reach}_{\mathcal{T}_k}(\{\min \mathcal{A}_{i*} \,|\, i \in \mathcal{A}_{*k}\})$. ∎

By definition, $\mathcal{V}_k = \mathcal{A}_{*k}^{[k-1]} \setminus \{1 \ldots k-1\}$. It can be computed using either of the next two theorems.

Theorem 5.8.

$$\mathcal{V}_k = \left(\bigcup_{k=parent(i)} \mathcal{V}_i \setminus \{i\} \right) \cup \{i \,|\, k = \min \mathcal{A}_{i*}\},$$

where each set in the above expression is disjoint from all other sets, and A has the strong Hall property. That is,

$$|\mathcal{V}_k| = \left(\sum_{k=parent(i)} |\mathcal{V}_i| - 1 \right) + |\{i \,|\, k = \min \mathcal{A}_{i*}\}|. \tag{5.2}$$

If A does not have the strong Hall property, this is an upper bound on $\mathcal{V}$.

The left-looking sparse QR algorithm `cs_qr` uses a less concise theorem to compute $\mathcal{V}_k$ to avoid the need for finding the set $|\{i \,|\, k = \min \mathcal{A}_{i*}\}|$.

Theorem 5.9.

$$\mathcal{V}_k = \left(\bigcup_{k=parent(i)} \mathcal{V}_i \setminus \{i\} \right) \cup (\mathcal{A}_{*k} \setminus \{1, \ldots, k-1\}), \tag{5.3}$$

where A has the strong Hall property. Otherwise, (5.3) *is an upper bound.*

These theorems are illustrated with an example matrix, its QR factorization in Figure 5.1, and its column elimination tree. The upper triangular matrix R and lower triangular matrix V containing the Householder vectors are shown as a single matrix. Fill-in entries that appear in R or V but not in A are shown as circled x's.

With these theorems, a left-looking sparse QR factorization algorithm can be stated in the following pseudocode, `sparse_qr_left`. It assumes $A_{kk}^{[k-1]}$ is struc-

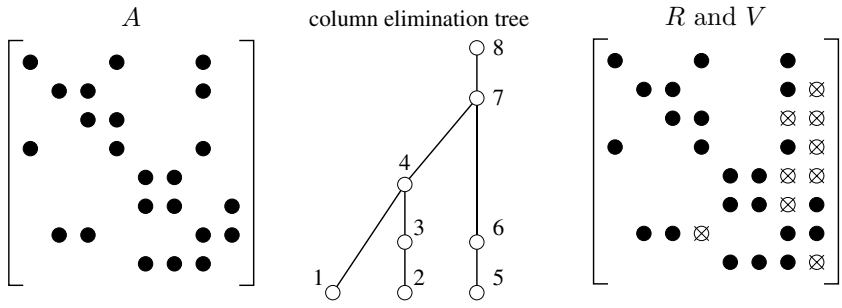


Figure 5.1. *QR factorization*

turally nonzero or, equivalently, $k \in \mathcal{V}_k$. Compare the following pseudocode function with the MATLAB function `qr_left`.

function $[V, \beta_{1\ldots n}, R] =$ `sparse_qr_left`(A)
  $\mathcal{T} =$ elimination tree of $A^T A$
  compute $|R|$ using `cs_counts` of $A^T A$
  compute $|\mathcal{V}_{1\ldots n}|$ using (5.2)
  for $k = 0$ to $n-1$ **do**
    $\mathcal{R}_{*k} = \mathrm{Reach}_{\mathcal{T}_k}(\{\min \mathcal{A}_{i*} \mid i \in \mathcal{A}_{*k}\})$
    $x = A_{*k}$
    $\mathcal{V}_k = \mathcal{A}_{*k}$
    for each $i \in \mathcal{R}_{*k}$ **do**
      $x = x - v_i(\beta_i(v_i^T x))$
      if `parent`$(i) = k$ **then**
        $\mathcal{V}_k = \mathcal{V}_k \cup \mathcal{V}_i \setminus \{i\}$
    $R_{1\ldots k-1,k} = x_{1\ldots k-1}$
    $[v_k, \beta_k, r_{kk}] =$ `house`$(x_{k\ldots m})$

The `cs_vcount` function evaluates (5.2). It also computes a row permutation P (represented by `pinv`) that ensures the diagonal entries $(PA)_{kk}^{[k-1]}$ are all structurally nonzero and finds `leftmost[i]` $= \min \mathcal{A}_{i*}$ for all rows i. It finds $|\mathcal{V}_{1\ldots n}|$ by creating and updating a set of n linked lists. At step k of the algorithm, list k will contain the set $\mathcal{V}_k$. The lists are initialized by placing each row i in the list $\min \mathcal{A}_{i*}$. The primary `for k` loop computes the row permutation and nonzero counts of V. In this loop, the first entry in the kth linked list is selected as the "pivot row"; this row becomes the kth row of PA to ensure that $(PA)_{kk}^{[k-1]}$ is structurally nonzero. If no such row exists, then the matrix A is structurally rank deficient and a fictitious row is created. Next, all rows in the kth linked list except this first row are removed from list k and placed in the linked list of the parent of k. After $|\mathcal{V}_{1\ldots n}|$ is computed, any unordered rows are assigned. It computes `S->lnz`, which is the number of entries in V, and `S->m2`, which is the number of rows in A after fictitious rows are added.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
static int cs_vcount (const cs *A, css *S)
{
    int i, k, p, pa, n = A->n, m = A->m, *Ap = A->p, *Ai = A->i, *next, *head,
        *tail, *nque, *pinv, *leftmost, *w, *parent = S->parent ;
    S->pinv = pinv = cs_malloc (m+n, sizeof (int)) ;        /* allocate pinv, */
    S->leftmost = leftmost = cs_malloc (m, sizeof (int)) ;  /* and leftmost */
    w = cs_malloc (m+3*n, sizeof (int)) ;   /* get workspace */
    if (!pinv || !w || !leftmost)
    {
        cs_free (w) ;                       /* pinv and leftmost freed later */
        return (0) ;                        /* out of memory */
    }
    next = w ; head = w + m ; tail = w + m + n ; nque = w + m + 2*n ;
    for (k = 0 ; k < n ; k++) head [k] = -1 ;   /* queue k is empty */
    for (k = 0 ; k < n ; k++) tail [k] = -1 ;
    for (k = 0 ; k < n ; k++) nque [k] = 0 ;
    for (i = 0 ; i < m ; i++) leftmost [i] = -1 ;
    for (k = n-1 ; k >= 0 ; k--)
    {
        for (p = Ap [k] ; p < Ap [k+1] ; p++)
        {
            leftmost [Ai [p]] = k ;         /* leftmost[i] = min(find(A(i,:)))*/
        }
    }
    for (i = m-1 ; i >= 0 ; i--)            /* scan rows in reverse order */
    {
        pinv [i] = -1 ;                     /* row i is not yet ordered */
        k = leftmost [i] ;
        if (k == -1) continue ;             /* row i is empty */
        if (nque [k]++ == 0) tail [k] = i ; /* first row in queue k */
        next [i] = head [k] ;               /* put i at head of queue k */
        head [k] = i ;
    }
    S->lnz = 0 ;
    S->m2 = m ;
    for (k = 0 ; k < n ; k++)               /* find row permutation and nnz(V)*/
    {
        i = head [k] ;                      /* remove row i from queue k */
        S->lnz++ ;                          /* count V(k,k) as nonzero */
        if (i < 0) i = S->m2++ ;            /* add a fictitious row */
        pinv [i] = k ;                      /* associate row i with V(:,k) */
        if (--nque [k] <= 0) continue ;     /* skip if V(k+1:m,k) is empty */
        S->lnz += nque [k] ;                /* nque [k] is nnz (V(k+1:m,k)) */
        if ((pa = parent [k]) != -1)        /* move all rows to parent of k */
        {
            if (nque [pa] == 0) tail [pa] = tail [k] ;
            next [tail [k]] = head [pa] ;
            head [pa] = next [i] ;
            nque [pa] += nque [k] ;
        }
    }
    for (i = 0 ; i < m ; i++) if (pinv [i] < 0) pinv [i] = k++ ;
    cs_free (w) ;
    return (1) ;
}
```

The `cs_sqr` function does the ordering and analysis for a sparse QR factor-

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

ization. Two parameters determine behavior of `cs_sqr`: `order` and `qr`. The `order` parameter specifies the ordering to use; the natural ordering (`order=0`) or a minimum degree ordering of A^TA (`order=3`) are good choices for a QR factorization. The `qr` parameter must be true (nonzero) for a sparse QR factorization. `cs_sqr` first finds a fill-reducing column permutation `S->q`. The function then finds the permuted matrix $C = AQ$ (where Q is the column permutation, not the orthogonal factor Q), determines the elimination tree of C^TC and postorders it, and finds the column counts of L (equivalently, the row counts of R). It then calls `cs_vcount` to find the column counts $|\mathcal{V}_{1\ldots n}|$ of the `V` matrix that holds the Householder vectors. The `cs_qr` function performs the numerical QR factorization.

```
css *cs_sqr (int order, const cs *A, int qr)
{
    int n, k, ok = 1, *post ;
    css *S ;
    if (!CS_CSC (A)) return (NULL) ;        /* check inputs */
    n = A->n ;
    S = cs_calloc (1, sizeof (css)) ;       /* allocate result S */
    if (!S) return (NULL) ;                 /* out of memory */
    S->q = cs_amd (order, A) ;              /* fill-reducing ordering */
    if (order && !S->q) return (cs_sfree (S)) ;
    if (qr)                                 /* QR symbolic analysis */
    {
        cs *C = order ? cs_permute (A, NULL, S->q, 0) : ((cs *) A) ;
        S->parent = cs_etree (C, 1) ;       /* etree of C'*C, where C=A(:,q) */
        post = cs_post (S->parent, n) ;
        S->cp = cs_counts (C, S->parent, post, 1) ;  /* col counts chol(C'*C) */
        cs_free (post) ;
        ok = C && S->parent && S->cp && cs_vcount (C, S) ;
        if (ok) for (S->unz = 0, k = 0 ; k < n ; k++) S->unz += S->cp [k] ;
        ok = ok && S->lnz >= 0 && S->unz >= 0 ;     /* int overflow guard */
        if (order) cs_spfree (C) ;
    }
    else
    {
        S->unz = 4*(A->p [n]) + n ;         /* for LU factorization only, */
        S->lnz = S->unz ;                   /* guess nnz(L) and nnz(U) */
    }
    return (ok ? S : cs_sfree (S)) ;        /* return result S */
}

csn *cs_qr (const cs *A, const css *S)
{
    double *Rx, *Vx, *Ax, *Beta, *x ;
    int i, k, p, m, n, vnz, p1, top, m2, len, col, rnz, *s, *leftmost, *Ap, *Ai,
        *parent, *Rp, *Ri, *Vp, *Vi, *w, *pinv, *q ;
    cs *R, *V ;
    csn *N ;
    if (!CS_CSC (A) || !S) return (NULL) ;
    m = A->m ; n = A->n ; Ap = A->p ; Ai = A->i ; Ax = A->x ;
    q = S->q ; parent = S->parent ; pinv = S->pinv ; m2 = S->m2 ;
    vnz = S->lnz ; rnz = S->unz ; leftmost = S->leftmost ;
    w = cs_malloc (m2+n, sizeof (int)) ;            /* get int workspace */
    x = cs_malloc (m2, sizeof (double)) ;           /* get double workspace */
    N = cs_calloc (1, sizeof (csn)) ;               /* allocate result */
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    if (!w || !x || !N) return (cs_ndone (N, NULL, w, x, 0)) ;
    s = w + m2 ;                                    /* s is size n */
    for (k = 0 ; k < m2 ; k++) x [k] = 0 ;          /* clear workspace x */
    N->L = V = cs_spalloc (m2, n, vnz, 1, 0) ;      /* allocate result V */
    N->U = R = cs_spalloc (m2, n, rnz, 1, 0) ;      /* allocate result R */
    N->B = Beta = cs_malloc (n, sizeof (double)) ;  /* allocate result Beta */
    if (!R || !V || !Beta) return (cs_ndone (N, NULL, w, x, 0)) ;
    Rp = R->p ; Ri = R->i ; Rx = R->x ;
    Vp = V->p ; Vi = V->i ; Vx = V->x ;
    for (i = 0 ; i < m2 ; i++) w [i] = -1 ; /* clear w, to mark nodes */
    rnz = 0 ; vnz = 0 ;
    for (k = 0 ; k < n ; k++)               /* compute V and R */
    {
        Rp [k] = rnz ;                      /* R(:,k) starts here */
        Vp [k] = p1 = vnz ;                 /* V(:,k) starts here */
        w [k] = k ;                         /* add V(k,k) to pattern of V */
        Vi [vnz++] = k ;
        top = n ;
        col = q ? q [k] : k ;
        for (p = Ap [col] ; p < Ap [col+1] ; p++)   /* find R(:,k) pattern */
        {
            i = leftmost [Ai [p]] ;         /* i = min(find(A(i,q))) */
            for (len = 0 ; w [i] != k ; i = parent [i]) /* traverse up to k */
            {
                s [len++] = i ;
                w [i] = k ;
            }
            while (len > 0) s [--top] = s [--len] ; /* push path on stack */
            i = pinv [Ai [p]] ;             /* i = permuted row of A(:,col) */
            x [i] = Ax [p] ;                /* x (i) = A(:,col) */
            if (i > k && w [i] < k)         /* pattern of V(:,k) = x (k+1:m) */
            {
                Vi [vnz++] = i ;            /* add i to pattern of V(:,k) */
                w [i] = k ;
            }
        }
        for (p = top ; p < n ; p++) /* for each i in pattern of R(:,k) */
        {
            i = s [p] ;                     /* R(i,k) is nonzero */
            cs_happly (V, i, Beta [i], x) ; /* apply (V(i),Beta(i)) to x */
            Ri [rnz] = i ;                  /* R(i,k) = x(i) */
            Rx [rnz++] = x [i] ;
            x [i] = 0 ;
            if (parent [i] == k) vnz = cs_scatter (V, i, 0, w, NULL, k, V, vnz);
        }
        for (p = p1 ; p < vnz ; p++)        /* gather V(:,k) = x */
        {
            Vx [p] = x [Vi [p]] ;
            x [Vi [p]] = 0 ;
        }
        Ri [rnz] = k ;                     /* R(k,k) = norm (x) */
        Rx [rnz++] = cs_house (Vx+p1, Beta+k, vnz-p1) ; /* [v,beta]=house(x) */
    }
    Rp [n] = rnz ;                          /* finalize R */
    Vp [n] = vnz ;                          /* finalize V */
    return (cs_ndone (N, NULL, w, x, 1)) ;  /* success */
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

The `cs_qr` function uses the symbolic analysis computed by `cs_sqr`: the column elimination tree `S->parent`, column preordering `S->q`, row permutation `S->pinv`, the `S->leftmost` array, the number of nonzeros in R and V (`S->unz` and `S->lnz`, respectively), and the number of rows `S->m2` after adding fictitious rows if A is structurally rank deficient.

The function first extracts the contents of `S`, allocates the result `N`, and allocates and initializes some workspace. Next, each column `k` of `V` and `R` is computed. The body of the `for k` loop first determines where `V(:,k)` and `R(:,k)` start, and finds the column `A(:,col)` corresponding to `C(:,k)`. The nonzero pattern $\mathcal{R}_{*k}$ of the `k`th column of `R` is found using a symbolic sparse triangular solve (Theorem 5.7). Prior Householder reflections are applied to column `k`, one for each nonzero entry in $|\mathcal{R}_k|$, and $\mathcal{V}_k$ is computed (5.3). The modified column $x = H_{k-1} \dots H_1 C_{*k}$ is gathered from its dense vector representation `x` as the `k`th column of `V`, and overwritten with the `k`th Householder vector. A complete symbolic and numeric QR factorization, including a fill-reducing column preordering, can be computed with `S=cs_sqr(3,A,1)` followed by `N=cs_qr(A,S)`.

In MATLAB, `[Q,R]=qr(A)` computes the QR factorization of `A`. The fill-reducing column permutation must be applied to `A` prior to calling `qr`. The MATLAB `qr` function is based on Givens rotations, not Householder reflections. It returns the orthogonal matrix `Q`, not the more compact representation of `V`, `Beta`, and `pinv` that `cs_qr` uses.

The `cs_qright` and `cs_qleft` M-files apply the Householder reflections (`V`, `Beta`, and `p` as computed by `cs_qr`) to the left or right of a matrix. `cs_qleft` is similar to `cs_happly`, except that it applies all of the Householder vectors.

```
function X = cs_qright (V, Beta, p, Y)
%CS_QRIGHT apply Householder vectors on the right.
%   X = cs_qright(V,Beta,p,Y) computes X = Y*P'*H1*H2*...*Hn = Y*Q where Q is
%   represented by the Householder vectors V, coefficients Beta, and
%   permutation p.  p can be [], which denotes the identity permutation.
%   To obtain Q itself, use Q = cs_qright(V,Beta,p,speye(size(V,1))).
%
%   See also CS_QR, CS_QLEFT.

[m n] = size (V) ;
X = Y ;
if (~isempty (p)) X = X (:,p) ; end
for k = 1:n
    X = X - (X * (Beta (k) * V (:,k))) * V (:,k)' ;
end

function X = cs_qleft (V, Beta, p, Y)
%CS_QLEFT apply Householder vectors on the left.
%   X = cs_qleft(V,Beta,p,Y) computes X = Hn*...*H2*H1*P*Y = Q'*Y where Q is
%   represented by the Householder vectors V, coefficients Beta, and
%   permutation p.  p can be [], which denotes the identity permutation.
%
%   See also CS_QR, CS_QRIGHT.

[m2 n] = size (V) ;
[m ny] = size (Y) ;
X = Y ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
if (m2 > m)
    if (issparse (Y))
        X = [X ; sparse(m2-m,ny)] ;
    else
        X = [X ; zeros(m2-m,ny)] ;
    end
end
if (~isempty (p)) X = X (p,:) ; end
for k = 1:n
    X = X - V (:,k) * (Beta (k) * (V (:,k)' * X)) ;
end
```

The Householder vectors stored in V are typically much sparser than the explicit representation of Q. Try this short experiment in MATLAB, which compares Q (with 38,070 nonzeros) and V (with only 3,906 nonzeros):

```
load west0479
q = colamd (west0479) ;
[Q,R] = qr (west0479 (:,q)) ;
[V,beta,p,R2] = cs_qr (west0479 (:,q)) ;
Q2 = cs_qright (V, beta, p, speye(size(V,1))) ;
```

5.4 Givens rotations

A *Givens rotation* is a 2-by-2 orthogonal matrix that can be applied to a 2-by-1 vector to zero out a selected entry. If a and b are scalars, c and s are selected so that

$$Gx = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}^T \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix},$$

where $r = \pm\|x\|_2$. The coefficients c and s are computed using the following `givens2` MATLAB function.

```
function g = givens2(a,b)
if (b == 0)
    c = 1 ; s = 0 ;
elseif (abs (b) > abs (a))
    tau = -a/b ; s = 1 / sqrt (1+tau^2) ; c = s*tau ;
else
    tau = -b/a ; c = 1 / sqrt (1+tau^2) ; s = c*tau ;
end
g = [c -s ; s c] ;
```

If applied to a 2-by-n sparse matrix X (equivalently, to two rows selected from a larger matrix), the nonzero patterns of the rows of GX are both equal to the union of the nonzero pattern of the rows of X, except that any one entry in GX can be selected to become zero.

5.5 Row-merge sparse QR factorization

A Givens-rotation-based QR factorization requires 50% more floating-point operations than a Householder-based QR factorization for a full matrix, but it has some

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

advantages in the sparse case. Rows can be ordered to reduce the work below that of the Householder-based sparse QR. The disadvantage of using Givens rotations is that the resulting QR factorization is less suitable for multifrontal techniques.

MATLAB uses Givens rotations for its sparse QR factorization. It operates on the rows of R and A. The matrix R starts out equal to zero but with enough space allocated to hold the final R. Each step of the factorization brings in a new row of A and eliminates its entries with the existing R until it is either all zero or it can be placed as a new row of R. The `qr_givens_full` algorithm for full matrices is shown below. It assumes the diagonal of A is nonzero. The innermost loop annihilates the a_{ik} entry via a Givens rotation of the incoming ith row of A and the kth row of R.

```
function R = qr_givens_full (A)
[m n] = size (A) ;
for i = 2:m
    for k = 1:min(i-1,n)
        A ([k i],k:n) = givens2 (A(k,k), A(i,k)) * A ([k i],k:n) ;
        A (i,k) = 0 ;
    end
end
R = A ;
```

For the sparse case, the rotation to zero out a_{ik} must be skipped if it is already zero. The entries k that must be annihilated correspond to the nonzero pattern $\mathcal{V}_{i*}$ of the ith row of the Householder matrix V, discussed in the previous section.

Theorem 5.10 (George, Liu, and Ng [95])**.** *Assume A has a zero-free diagonal. The nonzero pattern $\mathcal{V}_{i*}$ of the ith row of the Householder matrix V is given by the path $f \rightsquigarrow \min(i, r)$ in the elimination tree $\mathcal{T}$ of $A^T A$, where $f = \min \mathcal{A}_{i*}$ is the leftmost nonzero entry in the ith row, and r is the root of the tree.*

This method is illustrated in the `qr_givens` M-file below.

```
function R = qr_givens (A)
[m n] = size (A) ;
parent = cs_etree (sparse (A), 'col') ;
A = full (A) ;
for i = 2:m
    k = min (find (A (i,:))) ;
    if (isempty (k))
        continue ;
    end
    while (k > 0 && k <= min (i-1,n))
        A ([k i],k:n) = givens2 (A(k,k), A(i,k)) * A ([k i],k:n) ;
        A (i,k) = 0 ;
        k = parent (k) ;
    end
end
R = sparse (A) ;
```

The M-file is not meant as an efficient implementation. Since operating on the rows of a sparse matrix is very slow, the matrix is converted into a full matrix first. In an efficient implementation, only the ith row of A is held in a working array of

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

size n. The matrix R is allocated with enough space for each final row, but it starts out empty. The ith row of A is annihilated with rows k in the path $f \rightsquigarrow \min(i, r)$ until encountering an empty row k of R, at which point the elimination stops and the partially eliminated row of A becomes the kth row of R. If A has a zero-free diagonal, this happens when $i = k$. This method is called the *row-merge* QR factorization algorithm; it could also be called an up-looking sparse QR, since at the ith step it accesses only row i of A and rows 1 to $\min(i, n)$ of R. The `qr_givens` function assumes `A` has a zero-free diagonal.

5.6 Further reading

George and Heath [83] present the Givens-rotation row-merge algorithm; the `qr` function in MATLAB is Gilbert's implementation of this method. The row-merge scheme of Liu [149] is a generalization of the George–Heath method. George and Liu [90] compare Householder reflections and Givens rotations for sparse QR factorization. Heath [128] surveys a range of methods for solving sparse linear least squares problems. Givens [112] and Householder [138] discuss the transformations later named after them. Householder reflections, Givens rotations, QR factorization, and the Gram–Schmidt method are all discussed at length by Golub and Van Loan [114], Higham [135], and Stewart [190].

Many papers have considered the sparsity pattern of Q, R, and the Householder matrix V and data structures to represent them. Coleman, Edenbrandt, and Gilbert [22] showed that a sequence of symbolic Givens rotations gives a tight upper bound on the structure of R when A is strong Hall. George, Liu, and Ng [95] describe an efficient data structure for storing the Householder matrix V, using paths in the elimination tree. The Gram–Schmidt method is not typically used in the sparse case, since it computes Q explicitly, and an explicit sparse Q has many more nonzeros than the Householder matrix V. Ng and Peyton [158] describe an efficient data structure for an explicit sparse Q.

Gilbert, Ng, and Peyton [108] discuss the use of separators for predicting structure in sparse QR factorization. Hare et al. [127] show how to compute the nonzero pattern of Q and R. Pothen [168] extended these results by showing that there exist matrices that reach these bounds. Ng and Peyton [160] determine a bound on the nonzero pattern of Q based on the Householder matrix V. Gilbert et al. [102] and Gilbert, Ng, and Peyton [107] discuss how to compute the row and column counts for Cholesky, QR, and LU factorization and how to compute the elimination tree of $A^T A$. Oliveira [162] discusses how to predict the pattern of QR factorization without computing the Dulmage–Mendelsohn decomposition (see also Section 7.4). Many of these methods for determining the sparsity patterns for both QR and LU factorization are surveyed by Gilbert and Ng [106].

Row orderings have a large impact on the intermediate fill-in and total work required for the row-merge sparse QR, as shown by George and Ng [96], who present a nested dissection approach for determining a good row permutation. See also George, Liu, and Ng's three-part series [92, 93, 94] and Gillespie and Olesky [111].

Sparse multifrontal QR factorization is typically based on Householder reflec-

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

tions; see Lu and Barlow [154], Matstoms [156], Amestoy, Duff, and Puglisi [6], and Pierce and Lewis [167] (who also present an approximate rank-revealing multifrontal QR algorithm).

Underdetermined systems can be solved with QR factorization applied to A^T, as discussed by George, Heath, and Ng [84].

Exercises

5.1. Write a function that computes the nonzero pattern of `V` and `R`.

5.2. Modify `cs_qr` so that it can handle `m`-by-`n` matrices where `m` $<$ `n`. One simple solution is to append empty rows onto `A`, but this will not be efficient if `m` is much smaller than `n`.

5.3. Write a function `cs_reqr (cs *A, css *S, csn *N)` that computes a QR factorization. It should assume that the nonzero patterns of `V` and `R` are already computed.

5.4. Add column pivoting to `cs_qr`. If a column has a norm less than or equal to a given tolerance, permute it to the end of the matrix. The matrices `V` and `R` will need to be dynamically reallocated (see `cs_lu` in Chapter 6), since permuting the columns breaks the symbolic preanalysis.

5.5. Combine the postordering with the fill-reducing ordering in `cs_sqr` (see Problem 4.9 for details).